

Projekt polega na stworzeniu prostej aplikacji webowej opartej na mikroservisach. Celem aplikacji jest utworzenie mini-forum, w którym użytkownicy mogą dodawać własne komentarze, podpisane swoim nickiem. Przedstawi on jak można wykorzystać mikroservisy do utworzenia usługi webowej, z wykorzystaniem bazy danych Redis oraz usługi webowej Flask. Całość będzie działała w środowisku Docker, zarządzanym przez Docker Compose.

Mikroservisy

*//***Mikroservisy to podejście do tworzenia aplikacji, w którym duża aplikacja jest podzielona na mniejsze, niezależne komponenty.

Definicja zaczerpnięta z: <https://devstockacademy.pl/blog/narzedzia-i-automatyzacja/mikroservisy-jak-dzielic-aplikacje-na-mniejsze-czesci-i-dlaczego-warto-to-robic/>

***//*

Duże aplikacje są zbyt skomplikowane, żeby zarządzać nimi jako jedną całością. Dlatego dzieli się ją na mniejsze, niezależne części.

Każdy mikroservis działa jako odrębna jednostka, odpowiadająca za konkretną funkcję w systemie, co ułatwia zarządzanie, rozwój oraz testowanie aplikacji.

Źródło dodatkowe: <https://learn.microsoft.com/pl-pl/azure/architecture/guide/architecture-styles/microservices>

Docker

Docker to otwarta platforma umożliwiająca tworzenie, uruchamianie i zarządzanie aplikacjami, w odizolowanych kontenerach – lekkiej jednostce, zawierającej kod aplikacji oraz jej zależności, np biblioteki i konfigurację systemową. Możemy je udostępniać, mając pewność, że każda osoba otrzyma ten sam kontener, działający w ten sam sposób. *Dlatego został moim wyborem.* Źródło: <https://docs.docker.com/get-started/docker-overview/>

Docker Compose

Docker Compose to narzędzie do uruchamiania aplikacji wielokontenerowych. Ułatwia on zarządzanie usługami – za pomocą jednego pliku **docker-compose.yml** możemy tworzyć i uruchamiać wszystkie usługi.

Źródło: <https://docs.docker.com/compose/>

Wybrałem go, ponieważ umożliwia on zarządzanie całym zestawem kontenerów w sposób spójny i powtarzalny.

Flask

*/**Flask to lekki framework internetowy w języku Python, który pomaga szybko tworzyć aplikacje internetowe, zapewniając elastyczność bez zbędnej złożoności. Definicja zaczerpnięta z: <https://realpython.com/tutorials/flask/> **//*

Flask odpowiada za samą aplikację webową – stronę, którą widzi użytkownik. W moim projekcie, odpowiada on również za wyświetlanie komentarzy oraz przyjmowanie nowych wpisów z formularza, a następnie przesyłanie ich do bazy danych Redis.

Wybrałem go, ponieważ jest lekki, elastyczny i pozwala szybko tworzyć prototypy aplikacji webowych.

Redis

Redis to baza danych typu NoSQL. Pozwala na przechowywanie danych w formie key-value – każda wartość jest powiązana z unikalnym kluczem, pod którym można ją szybko odczytać lub zmienić. Jest on bardzo wydajny, ze względu na działanie w większości w pamięci RAM.

Źródło: <https://redis.io/learn/develop/node/nodecrashcourse/whatisredis>

W mojej aplikacji odpowiada ona za zapisywanie komentarzy pod odpowiednim kluczem. Wybrałem go ze względu na szybkość działania w pamięci operacyjnej, co umożliwia szybki odczyt i zapis danych.

Zastosowane środowisko i narzędzia

Projekt wykonałem na komputerze z systemem **Windows 11**, oraz z użyciem:

- **Docker Desktop** – do uruchamiania kontenerów Docker,
- **Visual Studio Code** – do edycji plików projektu i monitorowania logów.

Środowisko zostało w pełni wirtualizowane za pomocą **Dockera** i **Docker Compose**.

Test programu

Uruchomienie programu

Uruchomienie środowiska poleceniem ***docker compose build --no-cache*** oraz ***docker compose up -d***.

Docker compose powinien zbudować obraz web na podstawie *app/Dockerfile*, stworzyć i uruchomić usługi web i redis.

```
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose build --no-cache
[+] Building 6.2s (13/13) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 604B                                           0.0s
=> [internal] load build definition from Dockerfile                    0.0s
=> => transferring dockerfile: 242B                                       0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim    1.2s
=> [auth] library/python:pull token for registry-1.docker.io          0.0s
=> [internal] load .dockerignore                                        0.0s
=> => transferring context: 2B                                             0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:193fdd8bbcb3d2ae612bd6cc3548d2f7c78d65b549fcaa8af75624c47474444d 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:193fdd8bbcb3d2ae612bd6cc3548d2f7c78d65b549fcaa8af75624c47474444d 0.0s
=> [internal] load build context                                        0.0s
=> => transferring context: 93B                                           0.0s
=> CACHED [2/5] WORKDIR /app                                           0.0s
=> [3/5] COPY requirements.txt .                                        0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt           3.8s
=> [5/5] COPY . .                                                      0.0s
=> exporting to image                                                  0.9s
=> => exporting layers                                                    0.6s
=> => exporting manifest sha256:bda0d44795998cacfa12af7a10cb3462390db709ada2b409554f09643dcc229a 0.0s
=> => exporting config sha256:596bf4dab3d0773004884464b858ed5f2fa8ed1e8606e1918c995023810d6825 0.0s
=> => exporting attestation manifest sha256:347c972b1c4eca450ec8fc333288ea7fa44c8fbc83f0e288e7d9ffbbd3435ef 0.0s
=> => exporting manifest list sha256:f9805a8f5adf3c9ccb9f5557a0893d7febdcb1f37c71f286a65cf8bd0478261 0.0s
=> => naming to docker.io/library/project-docker-compose-web:latest 0.0s
=> => unpacking to docker.io/library/project-docker-compose-web:latest 0.2s
=> resolving provenance for metadata file                               0.0s
[+] Building 1/1
✓ project-docker-compose-web Built                                     0.0s
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose up -d
[+] Running 3/3
✓ Network project-docker-compose_default Created                    0.0s
✓ Container project-docker-compose-redis-1 Started                 0.4s
✓ Container project-docker-compose-web-1 Started                   0.5s
PS C:\Users\besmo\Desktop\project-docker-compose> |
```

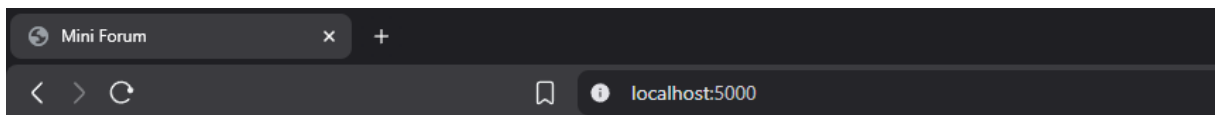
Oczekiwany wynik: Po sprawdzeniu statusu kontenerów docker compose powinien pokazać obu działające kontenery - w statusie Up.

```
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE  CREATED      STATUS      PORTS
project-docker-compose-redis-1      redis:7-alpine                     "docker-entrypoint.s..."          redis    8 seconds ago Up 7 seconds 6379/tcp
project-docker-compose-web-1        project-docker-compose-web         "python app.py"                     web      8 seconds ago Up 7 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS C:\Users\besmo\Desktop\project-docker-compose>
```

Test działania aplikacji w przeglądarce

Aplikacja działa na porcie 5000, więc po uruchomieniu możemy wejść w przeglądarce na stronę `http://localhost:5000`. Redis działa w tle na swoim wewnętrznym porcie 6379.

Oczekiwany wynik: Powinna się pojawić nasza strona -mini forum, z widocznymi formularzami oraz miejscem na dodane komentarze. Lista komentarzy powinna być pusta, ponieważ nie zapisywaliśmy jeszcze niczego w bazie redis. W przypadku wprowadzenia danych, powinny pokazać się zapisane wpisy.



Mini Forum

Twój nick Wpisz komentarz

Komentarze:

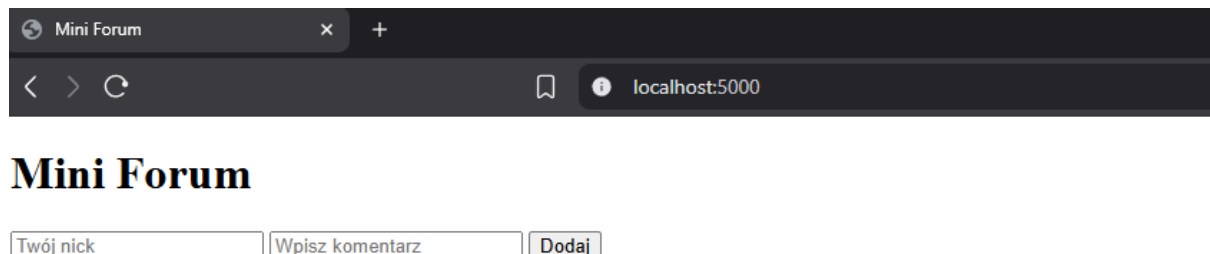
Wprowadzenie danych do formularza.

Wprowadzamy testowe dane do formularza i zatwierdzamy przyciskiem „Dodaj”.

Nick	Komentarz	➔ Dalej
Basia12	Witam i pozdrawiam	✓

Norbert G.	Dzień dobry!	✓
------------	--------------	---

Oczekiwany wynik: Po kliknięciu przycisku „Dodaj”, strona powinna automatycznie odświeżyć się i wyświetlić nowy komentarz na górze listy.



Komentarze:

- Dzień dobry! ~Norbert G.
- Witam i pozdrawiam ~Basia12

Sprawdzenie zawartości bazy Redis

Aby upewnić się, czy komentarze faktycznie zapisują się w bazie danych, a nie tylko wyświetlają po stronie aplikacji, możemy ręcznie podejrzeć zawartość pamięci Redis poleceniami ***docker compose exec redis redis-cli***, a następnie ***LRANGE comments 0 -1***.

Oczekiwany wynik: Redis powinien wyświetlić wszystkie komentarze dodane wcześniej z formularza, w formacie odpowiadającym wpisom widocznym na stronie.

Redis w swojej konsoli nie pokazuje polskich znaków jako liter, dlatego wprowadzona przez nas literka „ń” zostanie przedstawiona w bajtowej reprezentacji UTF-8.

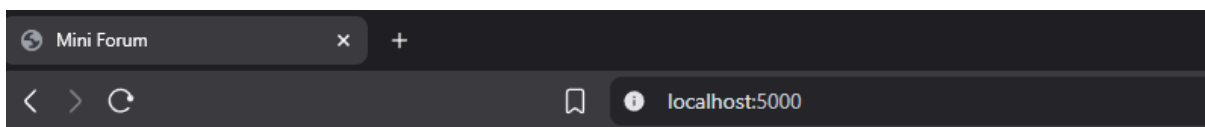
```
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose exec redis redis-cli
127.0.0.1:6379>
127.0.0.1:6379> LRANGE comments 0 -1
1) "Dzie\xc5\x84 dobry! ~Norbert G."
2) "Witam i pozdrawiam ~Basia12"
```

Test trwałości danych

Sprawdzamy działania bazy Redis – czy komentarze nie znikają po wyłączeniu aplikacji. Restartujemy aplikację poprzez ***docker compose down*** i ***docker compose up -d***.

Oczekiwany wynik: Po restarcie wszystkie komentarze powinny pozostać na stronie w niezmienionej formie i dalej być widoczne w pamięci Redis.

```
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose exec redis redis-cli
127.0.0.1:6379>
127.0.0.1:6379> LRANGE comments 0 -1
1) "Dzień\x05\x84 dobry! ~Norbert G."
2) "Witam i pozdrawiam ~Basia12"
127.0.0.1:6379> exit
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose ps
NAME                IMAGE              COMMAND              SERVICE  CREATED      STATUS      PORTS
project-docker-compose-redis-1  redis:7-alpine    "docker-entrypoint.s..."  redis    12 minutes ago  Up 12 minutes  6379/tcp
project-docker-compose-web-1    project-docker-compose-web  "python app.py"         web      12 minutes ago  Up 12 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose down
[+] Running 3/3
  ✓ Container project-docker-compose-web-1      Removed      1.3s
  ✓ Container project-docker-compose-redis-1    Removed      0.2s
  ✓ Network project-docker-compose_default      Removed      0.4s
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose up -d
[+] Running 3/3
  ✓ Network project-docker-compose_default      Created      0.0s
  ✓ Container project-docker-compose-redis-1    Started      0.3s
  ✓ Container project-docker-compose-web-1      Started      0.5s
PS C:\Users\besmo\Desktop\project-docker-compose> docker compose exec redis redis-cli
127.0.0.1:6379>
127.0.0.1:6379> LRANGE comments 0 -1
1) "Dzień\x05\x84 dobry! ~Norbert G."
2) "Witam i pozdrawiam ~Basia12"
127.0.0.1:6379>
```



Mini Forum

Komentarze:

- Dzień dobry! ~Norbert G.
- Witam i pozdrawiam ~Basia12

Podsumowanie testów

Wszystkie testy zostały wykonane pomyślnie. Aplikacja działa stabilnie, zapisuje dane, komunikuje się z Redisem i odzyskuje dane po ponownym uruchomieniu. Jest to zgodne z założeniami projektu.

Struktura projektu

```
PS C:\Users\besmo\Desktop\project-docker-compose> tree /F
Folder PATH listing for volume Sys
Volume serial number is 62B0-742C
C:
├── docker-compose.yml
└── app
    ├── app.py
    ├── Dockerfile
    └── requirements.txt
```

docker-compose.yml – Jest to plik konfiguracyjny Docker Compose. Definiuje wszystkie usługi aplikacji, czyli Flask i Redis, ustawia porty i zmienne.

app/app.py – Jest to plik z kodem aplikacji Flask, obsługuje formularz i komentarze. Łączy się z bazą Redis, zapisuje i wyświetla komentarze.

app/Dockerfile – Zawiera instrukcje do stworzenia obrazu Dockera dla aplikacji. Instaluje Pythona i potrzebne biblioteki, uruchamia app.py.

app/requirements.txt – Dokument tekstowy, który zawiera listę bibliotek potrzebnych do działania Flask i Redisa. Docker instaluje je przy budowie obrazu.

Wnioski końcowe

Projekt pokazuje praktyczne zastosowanie mikroserwisów do stworzenia prostej aplikacji webowej. Użycie Flask umożliwia szybkie stworzenie interfejsu użytkownika, a Redis zapewnia wydajne przechowywanie danych w pamięci operacyjnej, co pozwala nam na szybkie dodawanie i wyświetlanie komentarzy.

Dzięki Dockerowi i Docker Compose projekt jest w pełni wirtualizowany, co umożliwia łatwe uruchamianie aplikacji na różnych maszynach oraz szybkie jego odtwarzanie. Testy wykazały, że komentarze są poprawnie zapisywane

w Redisie i pozostają po restarcie kontenerów, co potwierdza prawidłową konfigurację oraz trwałość danych w pamięci.

Dalszy rozwój

Mój projekt – mini-forum aktualnie pozwala jedynie na dodawanie i wyświetlanie komentarzy. Aplikacja działa poprawnie, ale istnieje wiele możliwości jej dalszego rozwoju, aby zwiększyć jej funkcjonalność.

Obecnie możliwe jest wpisanie dowolnego nicku. W przyszłości można wprowadzić system logowania i rejestracji kont użytkowników oraz dodać rolę administratora, który będzie mógł usuwać nieodpowiednie komentarze.

Możemy również wprowadzić oznaczanie wiadomości tagami, co ułatwi nawigację po forum, wraz z opcją wyszukiwania po nicku, treści komentarza czy właśnie wybranego tagu.

Literatura:

- ❖ DevStock Academy. *Mikroserwisy – jak dzielić aplikacje na mniejsze części i dlaczego warto to robić*. <https://devstockacademy.pl/blog/narzedzia-i-automatyzacja/mikroserwisy-jak-dzielic-aplikacje-na-mniejsze-czesci-i-dlaczego-warto-to-robic/>
- ❖ Microsoft Docs. *Microservices architecture style*. <https://learn.microsoft.com/pl-pl/azure/architecture/guide/architecture-styles/microservices>
- ❖ Docker. *Get started with Docker*. <https://docs.docker.com/get-started/docker-overview/>
- ❖ Docker. *Docker Compose overview*. <https://docs.docker.com/compose/>
- ❖ Real Python. *Flask Tutorials*. <https://realpython.com/tutorials/flask/>
- ❖ Flask Official Documentation: <https://flask.palletsprojects.com/en/stable/>
- ❖ Redis. *What is Redis?*. <https://redis.io/learn/develop/node/nodecrashcourse/whatisredis>
- ❖ Redis Stack Documentation – Lists: <https://redis-stack.io/docs/data-types/lists/>

- ❖ Redis-py Documentation: <https://redis-py-doc.readthedocs.io/en/master/>